

STOCK MARKET SIMULATOR

Technical Documentation

Data Structures — Java Implementation

Project Name	Stock Market Simulator
Name	Saifullah Saeed
Teacher	Sir Bilal Bukhari
Date	May 2025

Project Overview:

The Stock Market Simulator is a console-based Java application that allows users to manage a portfolio of stocks. It demonstrates core data structure concepts through real-world financial operations including buying, selling, and tracking equities.

Feature	Data Structure Used
Stock storage & traversal	Singly Linked List
Activity / event logging	Queue (FIFO)
Undo last operation	Stack (LIFO)
Buy / Sell operations	LinkedList node update
Interactive CLI menu	do-while + switch-case

System Architecture

The application is structured around four main classes that work together. The diagram below shows how each component interacts.

Class Hierarchy & Relationships

Stock (Model)	Node (Wrapper)	LinkedList / Stack / Queue
Holds sname, sid, sprice, quantity fields and stockdetails() printer.	Generic wrapper pointing to Stock data and the next Node.	Three containers using Node as building block; each exposes its own API.

Data Structures — Detailed Breakdown

The Stock Class (Data Model)

The Stock class is the atomic unit of information. Every stock traded on the market is represented as a Stock object with four attributes.

- sname — Company / ticker name (String)
- sid — Unique numeric identifier (int)
- sprice — Current price in Rupees (int)
- quantity — Number of shares owned (int)

```
Stock.java - Data Model
1 class Stock {
2     String sname;
3     int sid;
4     int sprice;
5     int quantity;
6
7     public Stock(String sname, int sid,
8                   int sprice, int quantity) {
9         this.sname = sname;
10        this.sid = sid;
11        this.sprice = sprice;
12        this.quantity = quantity;
13    }
14
15    public void stockdetails() {
16        System.out.println("Name: " + sname +
17                           " | ID: " + sid +
18                           " | Price Rs.: " + sprice +
19                           " | Qty: " + quantity);
20    }
21 }
```

Figure 1 — Stock.java: Data model and pretty-printer

Singly Linked List (Stock Registry)

The LinkedList class maintains a chain of Node objects where each Node wraps a Stock. It is the primary data store for all market stocks. The list supports four core operations.

Method	Time Complexity	Description
<code>addStock()</code>	$O(1)$	Appends a new stock node at the tail — constant time due to tail pointer.
<code>deleteStock()</code>	$O(n)$	Traverses from head until the target name is found, then bypasses the node.
<code>searchStock()</code>	$O(n)$	Linear scan; prints full details if stock is found, otherwise notifies user.
<code>displayStock()</code>	$O(n)$	Iterates all nodes and calls <code>stockdetails()</code> on each.
<code>buyStock()</code>	$O(n)$	Finds the named stock and increments its quantity field.
<code>sellStock()</code>	$O(n)$	Decrements quantity after verifying sufficient shares exist.

```

LinkedList.java - Core Data Structure

1 class LinkedList {
2     Node head = null;
3     Node tail = null;
4
5     // Add stock to end of list
6     public void addStock(String sname, int sid,
7         int sprice, int quantity) {
8         Stock s = new Stock(sname, sid, sprice, quantity);
9         Node newNode = new Node(s);
10        if (head == null) { head = tail = newNode; }
11        else { tail.next = newNode; tail = newNode; }
12        System.out.println("Stock " + sname + " added!");
13    }
14
15    // Buy: increase quantity
16    public void buyStock(String name, int qty) {
17        Node t = head;
18        while (t != null) {
19            if (t.stockdata.sname.equalsIgnoreCase(name)) {
20                t.stockdata.quantity += qty;
21                System.out.println("Bought " + qty + " of " + name);
22                return;
23            }
24            t = t.next;
25        }
26        System.out.println("Stock not found!");
27    }
28
29    // Sell: decrease quantity
30    public void sellStock(String name, int qty) {
31        Node t = head;
32        while (t != null) {
33            if (t.stockdata.sname.equalsIgnoreCase(name)) {
34                if (t.stockdata.quantity >= qty)
35                    t.stockdata.quantity -= qty;
36                else
37                    System.out.println("Not enough shares!");
38                return;
39            }
40            t = t.next;
41        }
42    }
43 }

```

Figure 2 — *LinkedList.java: addStock(), buyStock(), and sellStock() implementations*

Stack (Undo History)

The Stack class implements a Last-In-First-Out structure to record every mutating action performed by the user. Each node stores the action type and a deep copy of the affected Stock's state, enabling precise reversal.

Action Type	Undo Behaviour
"ADD"	Deletes the stock that was added — effectively rolling back insertion.
"BUY"	Calls sellStock() with the saved quantity to reverse the purchase.
"SELL"	Calls buyStock() with the saved quantity to restore the shares.

```

Stack.java - Undo History
1 class Stack {
2     String type;           // "ADD", "BUY", "SELL"
3     Stock stockState;      // snapshot of the stock
4     Stack next;
5     public Stack top = null;
6
7     public Stack(String type, Stock stockToSave) {
8         this.type = type;
9         if (stockToSave != null)
10            this.stockState = new Stock(stockToSave.sname,
11                                         stockToSave.sid, stockToSave.sprice,
12                                         stockToSave.quantity);
13         else
14            this.stockState = null;
15         this.next = null;
16     }
17
18     // PUSH - save a new action
19     public void push(String type, Stock stockToSave) {
20         Stack newNode = new Stack(type, stockToSave);
21         newNode.next = top;
22         top = newNode;
23     }
24
25     // POP - retrieve and remove last action
26     public Stack pop() {
27         if (isEmpty()) return null;
28         Stack popNode = top;
29         top = top.next;
30         return popNode;
31     }
32
33     public boolean isEmpty() { return top == null; }
34 }

```

Figure

3 — *Stack.java: push(), pop(), and isEmpty() with deep-copy constructor*

Queue (Activity Log)

The Queue class follows the First-In-First-Out principle. Every user action (add, delete, buy, sell) is logged as a string message, providing a chronological audit trail. Internally it uses a private Node class distinct from the LinkedList's Node.

Method	Complexity	Purpose
<code>enqueue()</code>	$O(1)$	Appends a message string at the rear pointer — constant-time insertion.
<code>dequeue()</code>	$O(1)$	Removes the front node and advances the front pointer.
<code>display()</code>	$O(n)$	Prints every activity message from front to rear.

```

Queue.java - Activity Log
1 class Queue {
2     class Node {
3         String activity;
4         Node next;
5         public Node(String activity) {
6             this.activity = activity;
7             this.next = null;
8         }
9     }
10    Node front, rear;
11
12    // ENQUEUE - log a new activity
13    public void enqueue(String activity) {
14        Node n = new Node(activity);
15        if (front == null) { front = rear = n; }
16        else { rear.next = n; rear = n; }
17        System.out.println("Activity logged: " + activity);
18    }
19
20    // DEQUEUE - remove oldest activity
21    public void dequeue() {
22        if (front == null) {
23            System.out.println("Queue is empty");
24            return;
25        }
26        front = front.next;
27        if (front == null) rear = null;
28    }
29
30    // Display all activities
31    public void display() {
32        Node t = front;
33        while (t != null) {
34            System.out.println(t.activity);
35            t = t.next;
36        }
37    }
38 }

```

Figure 4 — Queue.java: enqueue(), dequeue(), and display()

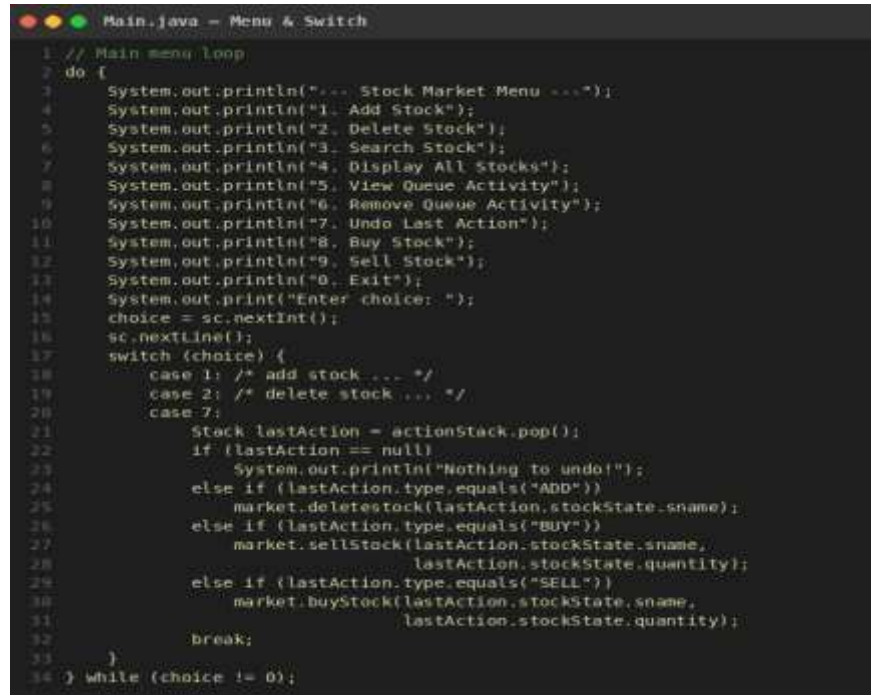
Main Class & Application Flow

The Mainn class is the entry point. It instantiates one LinkedList, one Queue, and one Stack, then enters a do-while loop that drives the menu. The switch statement routes each user choice to the appropriate data structure method and records the action for logging and undo support.

Menu Options

Option	Action	Data Structure(s) Involved
1	Add Stock	LinkedList (addStock) · Queue (log) · Stack (push ADD)
2	Delete Stock	LinkedList (deleteStock) · Queue (log)
3	Search Stock	LinkedList (searchStock)
4	Display All Stocks	LinkedList (displayStock)
5	View Activity Log	Queue (display)
6	Remove Activity	Queue (deque)
7	Undo Last Action	Stack (pop) → LinkedList (reverse operation)
8	Buy Stock	LinkedList (buyStock) · Queue (log) · Stack (push BUY)
9	Sell Stock	LinkedList (sellStock) · Queue (log) · Stack (push SELL)
0	Exit	Scanner closed; application terminates

Main Menu & Undo Logic (Code)

The image shows a screenshot of a code editor window titled "Main.java - Menu & Switch". The code is in Java and implements a main menu loop and an undo handler. The menu loop is a do-while loop that prints a menu of options (1-9 and 0 for exit) and prompts the user to enter a choice. A switch statement handles the choices. Case 7 is the undo handler, which pops the last action from an action stack. If the stack is empty, it prints "Nothing to undo!". Otherwise, it checks the action type: "ADD" (deletes the stock), "BUY" (sells the stock), or "SELL" (buys the stock). The code uses a Scanner for input and a Stack for the action history.

```
1 // Main menu loop
2 do {
3     System.out.println("--- Stock Market Menu ---");
4     System.out.println("1. Add Stock");
5     System.out.println("2. Delete Stock");
6     System.out.println("3. Search Stock");
7     System.out.println("4. Display All Stocks");
8     System.out.println("5. View Queue Activity");
9     System.out.println("6. Remove Queue Activity");
10    System.out.println("7. Undo Last Action");
11    System.out.println("8. Buy Stock");
12    System.out.println("9. Sell Stock");
13    System.out.println("0. Exit");
14    System.out.print("Enter choice: ");
15    choice = sc.nextInt();
16    sc.nextLine();
17    switch (choice) {
18        case 1: /* add stock ... */
19        case 2: /* delete stock ... */
20        case 7:
21            Stack lastAction = actionStack.pop();
22            if (lastAction == null)
23                System.out.println("Nothing to undo!");
24            else if (lastAction.type.equals("ADD"))
25                market.deleteStock(lastAction.stockState.sname);
26            else if (lastAction.type.equals("BUY"))
27                market.sellStock(lastAction.stockState.sname,
28                                lastAction.stockState.quantity);
29            else if (lastAction.type.equals("SELL"))
30                market.buyStock(lastAction.stockState.sname,
31                                lastAction.stockState.quantity);
32            break;
33    }
34 } while (choice != 0);
```

Figure 5 — Main.java: do-while menu loop and case 7 undo handler

Potential Improvements

The following enhancements would make the system more robust and production-ready.

Bug Fixes

- `deleteStock()` does not update the tail pointer when the tail node is deleted — can cause nullpointer exceptions on subsequent additions.
- `Stack` is a dual-purpose class (acts as both node and list container), which is an anti-pattern. Split into a `StackNode` class and a separate `ActionStack` class.
- `displayStock()` prints the counter number after the stock details instead of before it, giving incorrect numbering output.
- `sellStock()` does not print a confirmation message when the stock is not found (unlike `buyStock()`).

Feature Enhancements

- Persist data to a file (CSV or JSON) so stocks survive application restarts.
- Support fractional share prices using double or BigDecimal instead of int.
- Add a multi-level undo with configurable history depth, not just one level.
- Replace the Scanner-based CLI with a JavaFX or Swing GUI for a richer user experience.
- Introduce a Binary Search Tree or HashMap for $O(\log n)$ / $O(1)$ stock lookups instead of $O(n)$ linear scan.

1. Conclusion

The Stock Market Simulator successfully demonstrates three fundamental data structures — a Singly Linked List for persistent storage, a Queue for ordered activity logging, and a Stack for reversible operations — in a cohesive Java application. Each structure is hand-implemented without relying on the Java Collections Framework, making it an excellent learning vehicle for understanding pointer manipulation, memory layout, and algorithmic complexity.

The clean separation of concerns (Stock as a value object, Node as a generic wrapper, and three distinct container classes) provides a solid foundation for the enhancements outlined in Section 5.

Stock Market SiTmulator · Technical Documentation · v1.0

The End